

ARCMiS



Agentic Repository-Level Code Migration for Small Language Models

A hierarchical multi-agent harness that migrates legacy code between languages and proves every result with the toolchain. Coverage target now: C to Rust.

ARCMiS team

University of Engineering and Technology, Vietnam National University, Hanoi

hanhdd@vnu.edu.vn · 23021534@vnu.edu.vn

Repo: ARCMiS · docs/adr · docs/report · 2026-10-06

Contents

1. Introduction: what problem does ARCMiS solve?
2. Methodology: agents, rounds, gates, scoring, and the sweep
3. Inspirations: prior work this design builds on
4. Benchmarks: live campaign numbers
5. References

Introduction

What is code migration?

- Migration moves working code from one language to another.
- The new code must do the same job and pass the same tests.
- Many codebases still run on legacy C, Java, Go, and Python.
- Hand migration is slow. Each module needs the same steps: read, plan, translate, test.
- ARCMiS runs these steps with agents. The source and target languages are config, not code. The coverage sweep in this campaign runs C and Go sources to Rust targets.

```
/* C: raw pointer, raw length */
void scale(float* buf, int n, float k) {
    for (int i = 0; i < n; i++)
        buf[i] *= k;
}
```

```
// Rust: slice, bounds checked
fn scale(buf: &mut [f32], k: f32) {
    for x in buf.iter_mut() {
        *x *= k;
    }
}
```

Worked example, C to Rust (the campaign's demonstration case, not the project's scope). The pointer becomes a slice. Rust checks every access at compile time.

Why automate migration?

- Large codebases have hundreds of small modules.
- Each module needs the same steps: read, plan, translate, test.
- A single LLM call cannot hold a whole repository.
- An agent can call tools: read files, write files, run builds.
- ARCMiS coordinates many agents to cover one module set after another.

Step	Done by	Checked by
Read and map the source	agents	source map on disk
Plan the port	agents	plan file on disk
Translate to the target language	agents	target build command
Prove	agents	target test command

Any source language, any target

Why multi-agent?

One agent alone fails in known ways:

- It forgets the plan when the context fills up.
- It claims success without running the toolchain.
- One long task burns the whole time budget.
- A dead end (bad port idea) has no second opinion.

The ARCMiS answers:

Division of labor Each role has one job and its own context.

Delegation The orchestrator dispatches. Leads check the work.

Evidence gates A claim counts only with a file or a green test behind it.

The agent cast

Role	Tier	Tools	Output
orchestrator	1	none: DECISION grammar only	dispatches, phase exits
discovery-lead	2	dispatch, read	source map, brief
migration-lead	2	dispatch, read	translated batches
integration-lead	2	dispatch, read	merged tree, test runs
analyst, architect	3	read, find, write	maps, briefs
translator, repairer	3	read, write, edit	target-language source
validator, tester	3	read, target build/test	pass or fail reports
planner, failure-analyst, critic	3	read, write	plans, triage notes

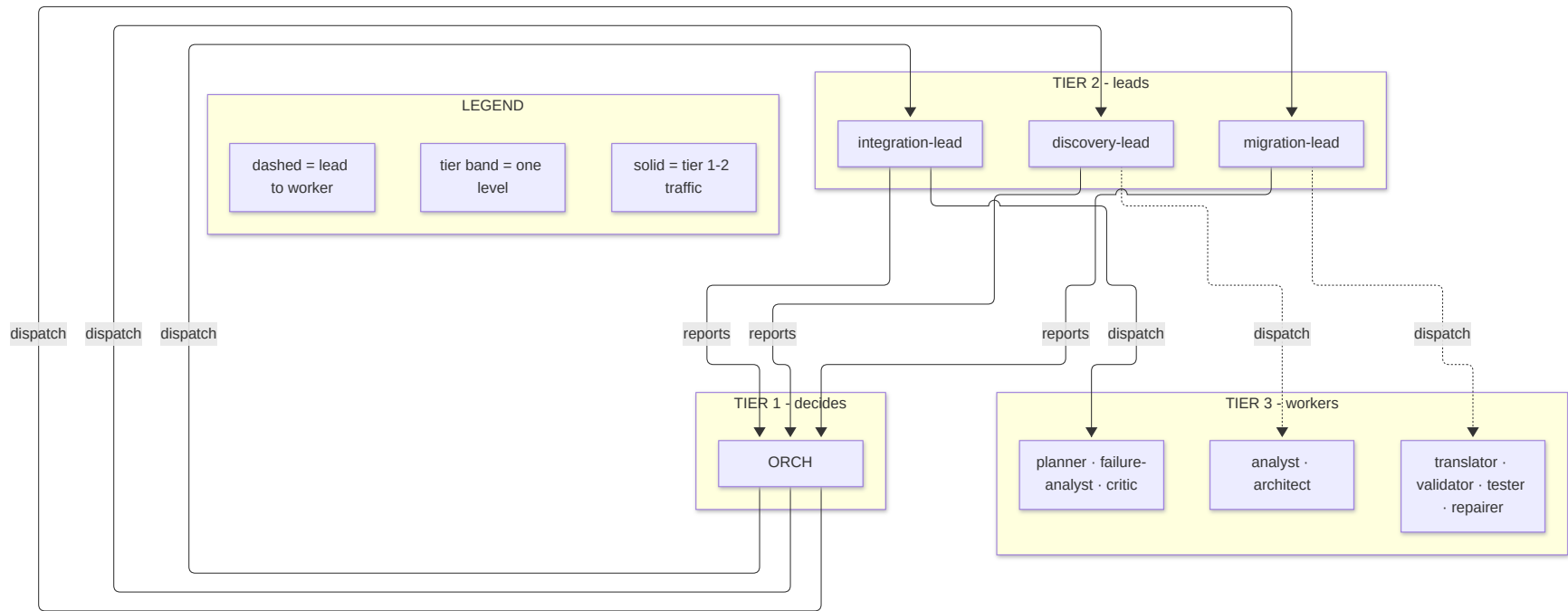
- Tier 1 decides. Tier 2 leads one work area each. Tier 3 runs the tools.
- Config-owned team split: `ARCMiS/lib/orchestrator/src/hierarchy.rs` .

Words this deck uses

Word	Meaning
round	one decision cycle: dispatch, results, next decision
phase	one migration stage: Discovery to Migration
gate	an evidence check before a phase exit
breaker	a stop rule for runs without progress
delegation	one dispatch of a task to a lead or specialist
candidate	one harness config under test
problem set	one project to migrate (one directory)
ledger	one row per round in <code>ROUNDS.yaml</code>
verdict	the scored outcome of one round

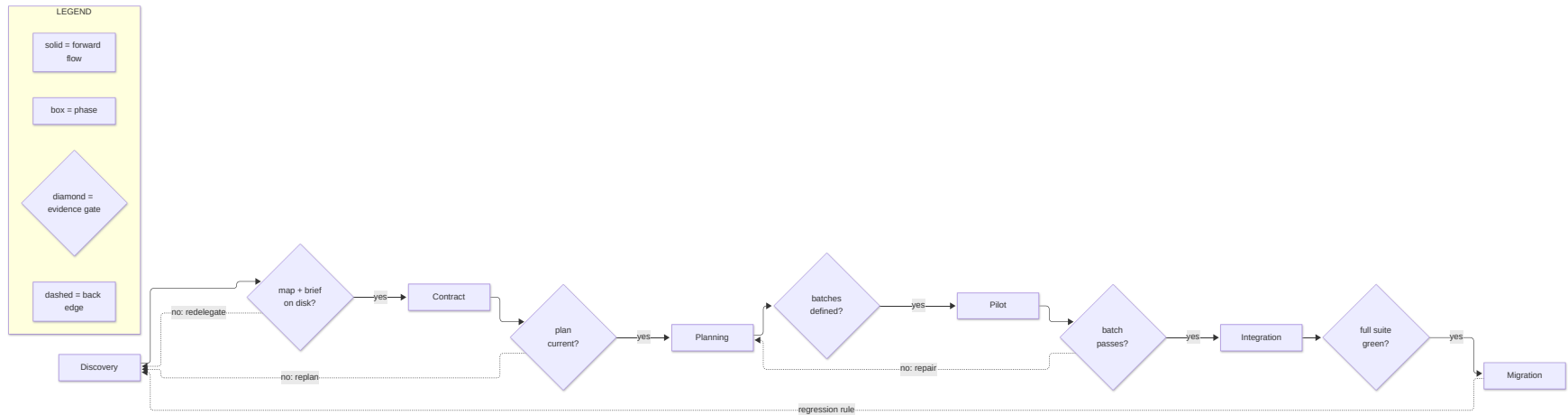
Methodology

The MAS hierarchy



- Tier 1 owns the round loop and the task graph.
- Tier 2 splits work and checks acceptance.
- Tier 3 never talks to tier 1 directly.

Life of a round: the state machine



- Each diamond asks for evidence on disk, not for a claim.

What each phase must prove

Phase	Exit evidence (the gate checks it)
Discovery	<code>source-map.md</code> + <code>brief.md</code> , every module cited
Contract	target architecture + plan, current on disk
Planning	translation batches, each with an acceptance check
Pilot	first batch translated, <code>cargo build</code> green
Integration	full tree merged, <code>cargo test</code> 0 failed
Migration	final validation, workspace scored by the toolchain

- A phase exit without the file is refused by the gate.
- Failed gates send the round back, not forward.

How a round works, step by step

1. The orchestrator reads the plan and the task graph.
2. It writes one short task with the DECISION grammar.
3. It dispatches the task to a lead.
4. The lead splits the task and dispatches specialists.
5. Specialists run tools and write artifacts.
6. The lead checks the acceptance clause and reports.
7. The gate checks phase evidence. Pass: next phase. Fail: back edge.
8. The ledger records the round: verdict, counters, artifacts.
 - One round = steps 1 to 8. A run = many rounds through 6 phases.
 - Rounds continue until a gate exits the last phase or a breaker stops the run.

Safety rails: gates and breakers

A run can waste hours. These rules stop it:

Rail	Trigger	Action
evidence gate	exit claim without files	refuse the exit
stagnation breaker	3 rounds, no completed task	close the run
round cap	20 rounds	close the run
MaxTurns limit	member exceeds its turn budget	member dies, lead retries
output cap	answer over 8192 tokens	retry with a tighter task
engine 503	engine overloaded	backoff, then retry

- Every trip lands in the ledger row for that round.

How scoring works

`scripts/rescore.py` runs after every round. No model self-grading.

1. Run the target build command on the produced workspace.
2. Run the target test command on the same workspace.
3. Count: tests passed, tests failed. Write `result/per_problem.json`.

Verdict	Meaning
SOLVED	build green, tests pass, round complete
NOT CLEARED	run closed without a verified workspace
ABORTED	engine or launcher failure, no verdict earned
IN FLIGHT	round still running

- Only the toolchain decides. A model claim is never a verdict.
- Campaign instantiation: the commands are `cargo build` and `cargo test`. The scoring path is demonstrated on Rust targets in this campaign.

Scoring generality: one scorer, many toolchains

What the ledger records

Every round writes one row to `ROUNDS.yaml` :

Field group	Fields
identity	round id, protocol, candidate id
intent	hypothesis, defect class targeted
cost	wall seconds, stalled rounds
deaths	max-turns deaths, output-cap deaths
result	compile, tests passed, tests failed
lineage	parents, delta vs parent, regression flags
outcome	verdict, artifact paths

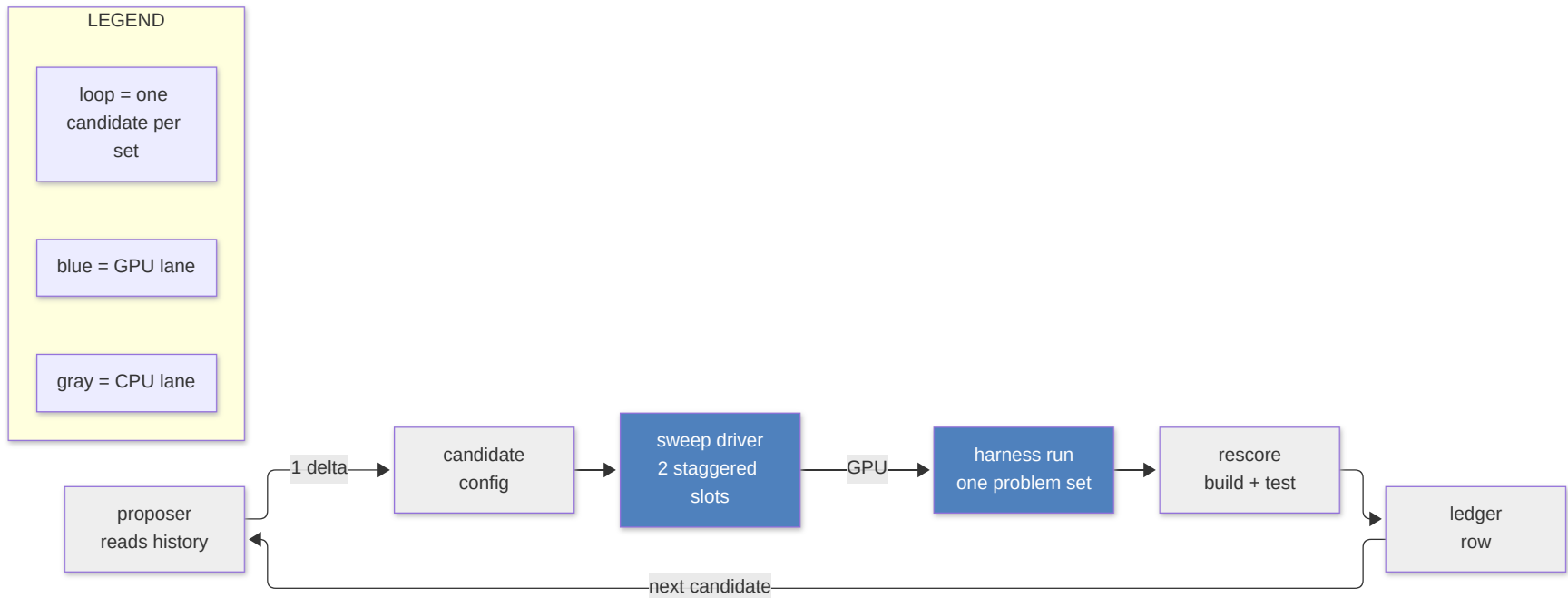
- The ledger is the campaign memory. Every retry keeps its own row.

Reading the results

- A problem clears when one round scores a verified workspace.
- A problem counts once across retries.
- NOT CLEARED may retry with a fix or a fresh config.
- ABORTED means no verdict. The retry starts from the same state.
- The scoreboard counts problems, not rounds.

Example: run s31 spent **14.7 h** on remimu, finished 12 of 13 tasks, hit the round cap, and scored NOT CLEARED. The ledger row keeps the full counter set, so the retry starts from a known state.

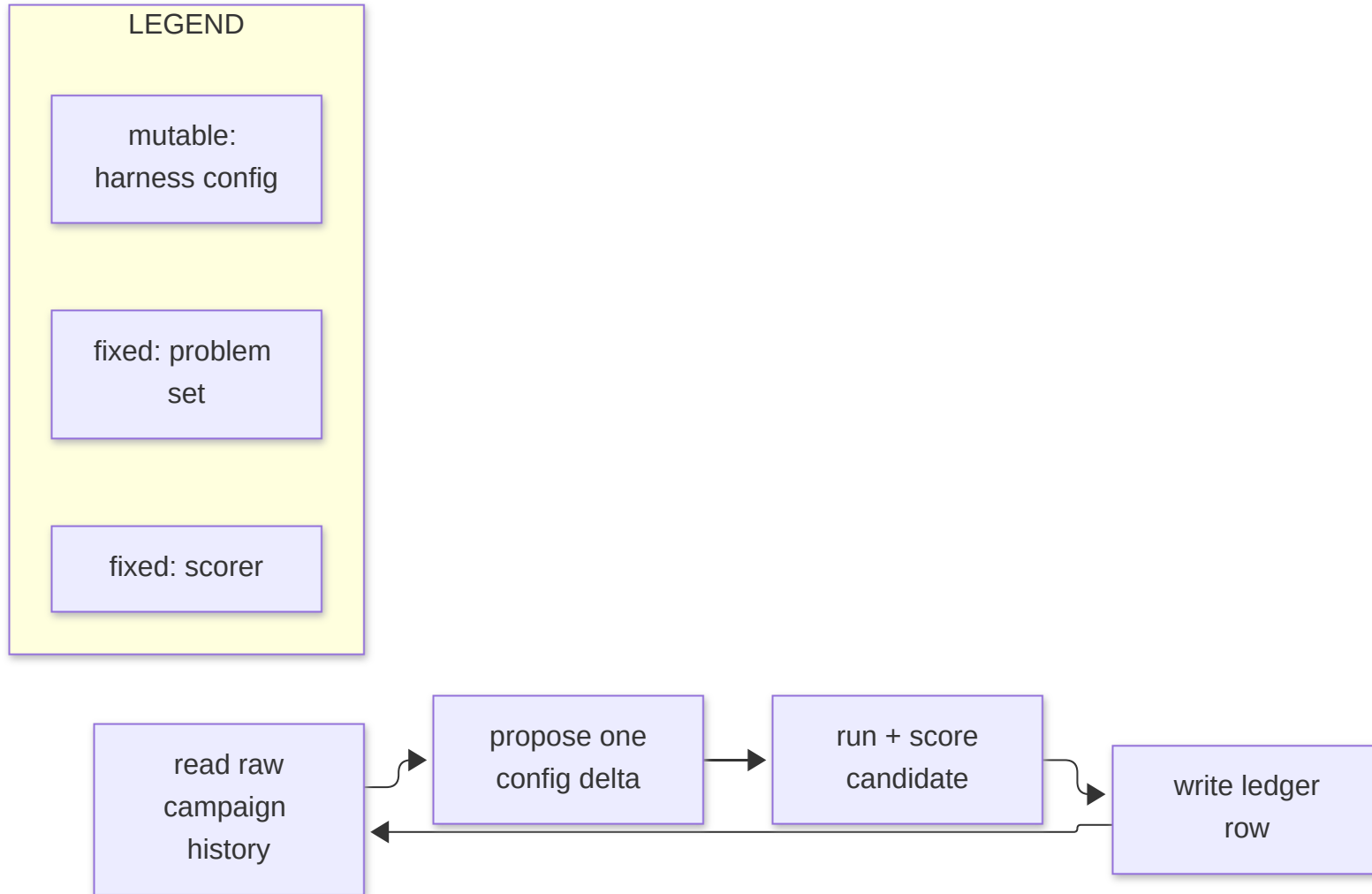
The coverage sweep pipeline



- GPU lane: one set at a time, LoC ascending, two staggered slots.
- CPU lane: proposer and scorer run while the GPU works.
- Coverage rule: untouched projects only. Pairs mix one crust set with one non-crust set.
- Campaign instantiation: Rust targets with the cargo toolchain.

The harness evolution loop

The harness itself changes between candidates:



- The problem set and the scorer stay fixed.
- One candidate changes one thing, so the ledger shows cause and effect.

Inspirations

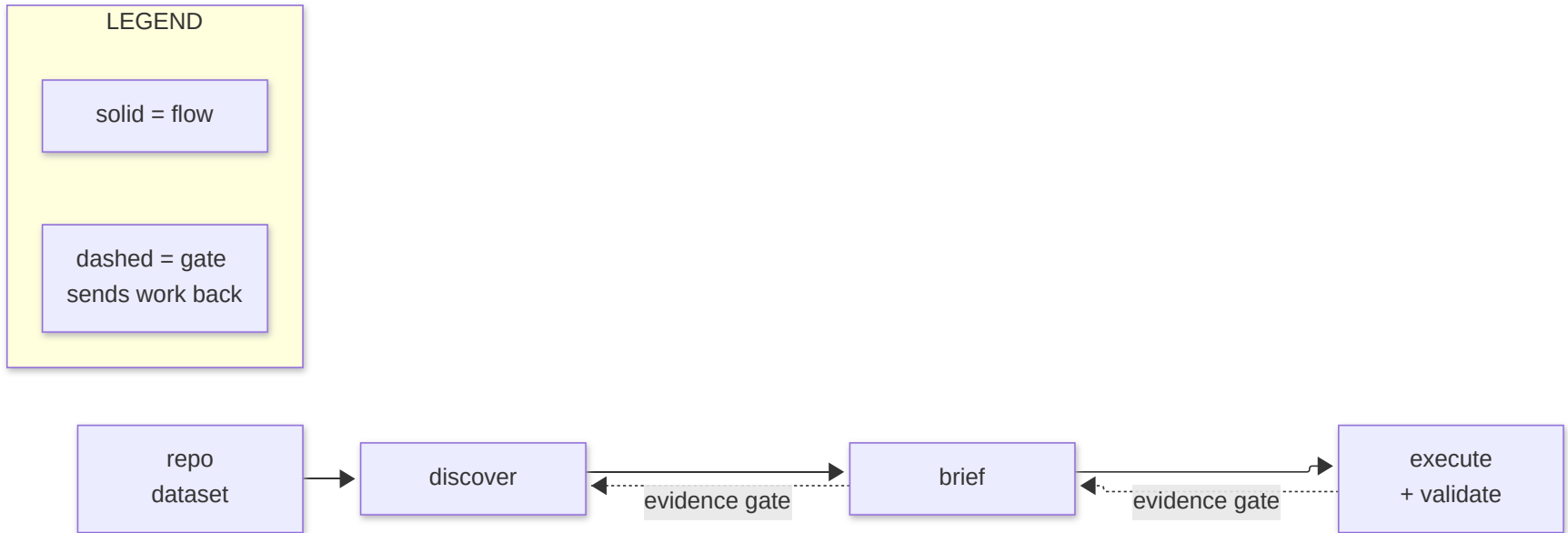
Orchestration patterns: the two axes

Source: the LLM multi-agent orchestration survey (the "Q2 survey").

	static agents	dynamic-adaptive
centralized	one boss, fixed roles	one boss, routing at runtime
decentralized	peer teams, fixed links	peers that renegotiate
hierarchy	fixed tree of teams	ARCMiS sits here

- Hierarchy: tier-2 leads over tier-3 specialists. Verified in `hierarchy.rs` and `lead.rs`. Recorded in ADR 0022 and ADR 0026.
- Dynamic: the task graph re-scores after each delegation, and the orchestrator can spawn a failure-analyst chain mid-task.
- Static part: fixed role registry, fixed preambles, fixed tool allowlists. The selection is dynamic. The agents are not.

Modernization workflows



ReCodeAgent (arXiv:2604.07341, ASE 2026):

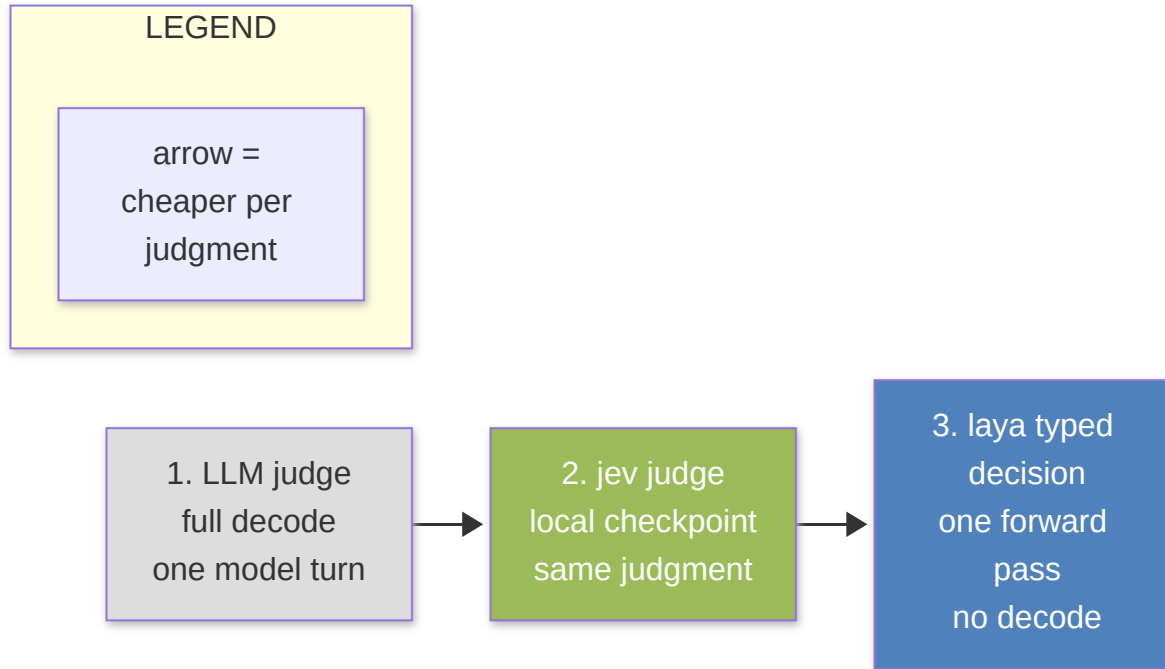
- A multi-agent workflow for repository translation and validation.
- ARCMiS borrows the `tool_projects` dataset and the stage discipline (ADR 0018, ADR 0022).

Claude Modernization Plugin:

- Discovery, brief, and execution flow with evidence-gated phase exits.
- ARCMiS borrows the gate idea: an exit needs evidence, not a claim.

System-one models: jev and laya

Judgments cost model turns. Three cheaper rungs exist:



- LLM judge: one full decode per judgment, on the busy GPU.
- jev judge: the same judgment from a local specialist checkpoint. `jev_judge.rs` (ADR 0023), `jev_triage.rs` (ADR 0027).
- laya (arXiv:2609.26550): one forward pass answers `choice`, `score`, and `noul`. See `.omp/skills/laya`.

Where triage is wired today

- Each member dispatch gets a typed triage verdict. Five `agent_trace_observability` questions, min-confidence cascade.
- Each round gets a laya consultation over its own counters (ADR 0028).

Honest state: the round policy defaults to `observe ( round_triage/policy.rs )`. The harness records every verdict. No round stops on one. Enforcement (`policy: enforce`) exists and is not the campaign default.

- Measured round confidences sit near 0.31 to 0.33, below the 0.9 cascade threshold. The judge falls back on today's data. Reported as measured. Not tuned to force agreement.

Benchmarks

The dataset

`assets/ReCodeAgent/data/tool_projects` , four families:



- Counted on disk: $4 + 100 + 7 + 8 = \mathbf{119}$ directories.
- The campaign brief says **114**. The delta is 5 problems.
- Slides below use the brief number 114 and say so.

Progress: 13 of 114 problems cleared



13 green = one verified workspace per problem. Each dot is one problem.

Family	Problems	Cleared
crust	100	11
oxidizer	7	1
skel	8	1
alphatrans	4	0

- A problem counts once across retries.
- The cleared set includes python-source (colorsys) and go-source (gonameparts) problems. All scored as Rust-target rounds.
- Live: wave 3 of the coverage sweep, 1 round in flight (2026-10-06).

Round verdicts so far

55 ledger rows, counted live from `ROUNDS.yaml` (2026-10-06):



- One problem can span several rounds. A retry keeps its own row.
- ABORTED rows are engine or launcher failures, not harness verdicts.

Wall clock: solved runs vs failed runs

Serial-era solved median: **145 min** (n=15). Three wave-2 failures:



green = solved run · red = not cleared · dashed line = solved median

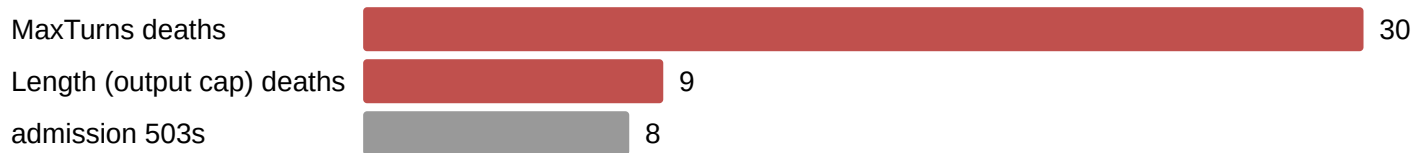
- A failure can end fast (fft, stuck at Planning) or run long (remimu, 5 of 5 batches translated, then the round cap).
- remimu took 6 times the median and still did not clear.

Where the tokens go: one failed run

The remimu round (s31, 14.7 h, NOT CLEARED at the round cap):



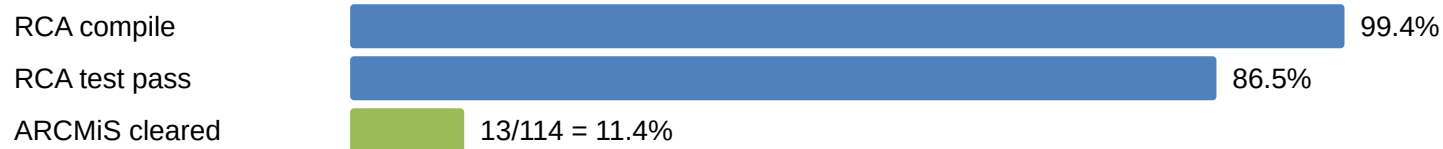
Deaths along the way (red = run-stopping class):



Legend: blue = tokens, purple = wasted output, red = member deaths, gray = engine overload.

Comparison with ReCodeAgent

	ReCodeAgent	ARCMiS
model	Claude 4.5 Sonnet	qwen3.8-27B, local
protocol	their developer-test suites	toolchain-only scoring
scale	118 projects	one set at a time



Different protocol: ARCMiS has no aggregate compile percent. The green bar is problems cleared under a stricter, toolchain-only gate. Directional context, not a like-for-like comparison.

Estimated time to finish

- Campaign start: 2026-09-26. Elapsed: 10 days (2026-10-06).
- Rate so far: 13 cleared / 10 days = about 1.3 per day.
- Remaining: $(114 - 13) / 1.3 =$ about **78 days**.

This is an estimate from the current rate, not a commitment. The rate changes with problem size. The LoC walk now reaches larger sets, so later problems cost more per set.

References

References

- ReCodeAgent: A Multi-agent Workflow for Language-Agnostic Translation and Validation of Large-Scale Repositories. arXiv:2604.07341, ASE 2026.
- LLM-Based Multi-Agent Orchestration: A Survey of Frameworks, Communication Protocols, and Emerging Patterns (the Q2 survey).
- Typed decisions. arXiv:2609.26550. `laya-rs` crate, `.omp/skills/laya` .
- ARCMiS ADRs 0018, 0022, 0023, 0026, 0027, 0028, 0029 in `docs/adr/` .
- Dataset: `assets/ReCodeAgent/data/tool_projects` .
- Ledger: `.artifacts/experiments/ROUNDS.yaml` and `SUMMARY.md` .

